UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+

Student Name: Yash Karde **UID: 25MCI10090**
Branch: MCA(AI/ML) **Section/Group: 25MAM-1 A**
Semester: 2nd sem **Date of Performance: 31/03/2026**
Subject Name: TECHINCAL SKILLS **Subject Code:**

WORKSHEET 7

AIM: Implementation of jOINS in PostgreSQL

S/W Requirement: Oracle Database Express Edition and pgAdmin

OBJECTIVES:

Apply joins to a real-world database schema (e.g., Students, Courses, Enrollments, Departments)

Practical / Experiment Steps

Table creation:

```
CREATE TABLE Students
   ( student_id INT PRIMARY KEY,
   student_name VARCHAR(50),
   dept_id INT
);

CREATE TABLE Courses
   ( course_id INT PRIMARY KEY,
   course_name VARCHAR(50)
);

CREATE TABLE Enrollments
   ( student_id INT,
   course_id INT
);

CREATE TABLE Departments
   ( dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50)
);


-- Students (added duplicate dept_id for SELF JOIN)
INSERT INTO Students VALUES (1, 'Amit', 101);
INSERT INTO Students VALUES (2, 'Sara', 102);
INSERT INTO Students VALUES (3, 'John', 103);
INSERT INTO Students VALUES (4, 'Neha', NULL);
INSERT INTO Students VALUES (5, 'Pankaj', NULL);
INSERT INTO Students VALUES (6, 'Rahul', 101);
INSERT INTO Students VALUES (7, 'Simran', 101);
```

```
-- Courses
INSERT INTO Courses VALUES (201, 'DBMS');
INSERT INTO Courses VALUES (202, 'Java');
INSERT INTO Courses VALUES (203, 'Python');

-- Enrollments
INSERT INTO Enrollments VALUES (1, 201);
INSERT INTO Enrollments VALUES (1, 202);
INSERT INTO Enrollments VALUES (2, 201);
INSERT INTO Enrollments VALUES (6, 203);

-- Departments
INSERT INTO Departments VALUES (101, 'CS');
INSERT INTO Departments VALUES (102, 'IT');
INSERT INTO Departments VALUES (103, 'ECE');
```

```
--- inner JOIN
SELECT s.student_id, s.student_name, c.course_name
FROM Students s
INNER JOIN Enrollments e
   ON s.student_id = e.student_id
INNER JOIN Courses c
   ON e.course_id = c.course_id;
```

Data Output Messages Notifications

	student_id integer	student_name character varying (50)	course_name character varying (50)
1	1	Amit	DBMS
2	1	Amit	Java
3	2	Sara	DBMS
4	6	Rahul	Python

```
--- left JOIN
SELECT s.student_name
FROM Students s
LEFT JOIN Enrollments e
   ON s.student_id = e.student_id
WHERE e.course_id IS NULL;
```

Data Output Messages Notifications

	student_name character varying (50)
1	Pankaj
2	Neha
3	John
4	Simran

```
--- RIGHT JOIN
SELECT c.course_name, s.student_name
FROM Courses c
LEFT JOIN Enrollments e
   ON c.course_id = e.course_id
LEFT JOIN Students s
   ON e.student_id = s.student_id;
```

Data Output Messages Notifications

	course_name character varying (50)	student_name character varying (50)
1	DBMS	Amit
2	Java	Amit
3	DBMS	Sara
4	Python	Rahul

```
--- multiple join
SELECT
   s.student_name,
   d.dept_name,
   c.course_name
FROM Students s
LEFT JOIN Departments d
   ON s.dept_id = d.dept_id
LEFT JOIN Enrollments e
   ON s.student_id = e.student_id
LEFT JOIN Courses c
   ON e.course_id = c.course_id;
```

Data Output Messages Notifications

	student_name character varying (50)	dept_name character varying (50)	course_name character varying (50)
1	Amit	CS	DBMS
2	Amit	CS	Java
3	Sara	IT	DBMS
4	Rahul	CS	Python
5	Pankaj	[null]	[null]
6	Neha	[null]	[null]
7	John	ECE	[null]
8	Simran	CS	[null]

```
--- self JOIN
SELECT A.student_name AS Student1, B.student_name AS Student2, A.dept_id
FROM Students A
JOIN Students B
   ON A.dept_id = B.dept_id
WHERE A.student_id < B.student_id;
```

Data Output Messages Notifications

Delete
Alt Shift D

	student1 character	student2 character varying (50)	dept_id integer
1	Amit	Simran	101
2	Amit	Rahul	101
3	Rahul	Simran	101

```
--- cross JOIN
SELECT s.student_name, c.course_name
FROM Students s
CROSS JOIN Courses c
LIMIT 10;
```

Data Output Messages Notifications

	student_name character varying (50)	course_name character varying (50)
1	Amit	DBMS
2	Sara	DBMS
3	John	DBMS
4	Neha	DBMS
5	Pankaj	DBMS
6	Rahul	DBMS
7	Simran	DBMS
8	Amit	Java
9	Sara	Java
10	John	Java

Outcomes:

1. INNER JOIN returns only matching records between tables (students who are enrolled in courses).
2. LEFT JOIN helps identify unmatched data (students with no course enrollments).
3. RIGHT JOIN ensures all records from the right table appear, even without matches.
4. SELF JOIN compares rows within the same table (students in the same department).
5. CROSS JOIN produces all possible combinations of rows (Cartesian product).